

BITS PILANI — WORK INTEGRATED LEARNING PROGRAMMES

Diabetes Risk Prediction Using Health and Lifestyle Indicators

Course: AIMLCZG549 — API-Driven Cloud-Native Solutions

Assignment I — Group 49

Pushadapu Sanjay Kumar · Sathish Krishnan V
Chezrla Raga Suma · Bhuvnesh Mishra

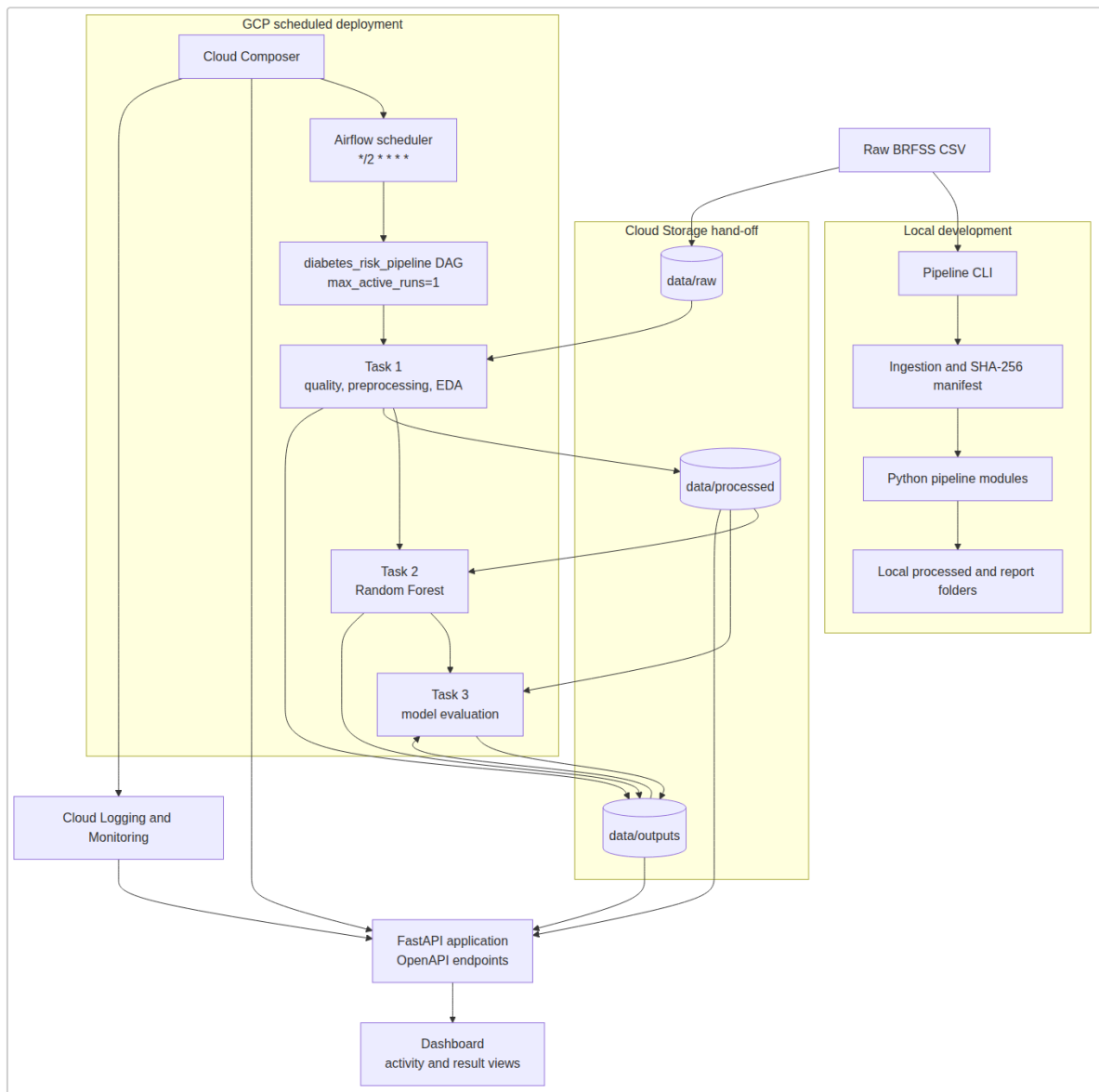
23 September 2026

Course: AIMLCZG549 — API-Driven Cloud-Native Solutions **Assignment:** Assignment I **Group:** 49
Institution: BITS Pilani, Work Integrated Learning Programmes

This report documents the design, implementation, and verification of a cloud-based, API-accessible data pipeline for diabetes-risk screening from health and lifestyle indicators, delivered as four work packages: (1) business understanding, dataset selection, and ingestion; (2) data-quality validation, preprocessing, and scheduled pipeline automation; (3) exploratory data analysis and an optional predictive-model comparison; and (4) dashboard, APIs, and demonstration.

Architecture Overview

The solution separates platform-neutral data processing from cloud orchestration and presentation. The same Python pipeline can run locally for development or inside Apache Airflow on Cloud Composer for scheduled execution. Pipeline outputs are written to local folders during development and to the corresponding Cloud Storage locations in the deployed environment.



Architecture overview: local development pipeline and GCP scheduled deployment, showing Cloud Composer orchestrating three tasks, Cloud Storage hand-off between them, and the FastAPI application and dashboard consuming the results

Execution steps

1. **Select the source:** use the immutable BRFSS CSV from `data/raw/` locally or the matching `data/raw/` object in Cloud Storage when running on Composer.
2. **Run ingestion locally:** the CLI validates the schema and row count, then appends a timestamped SHA-256 record to the ingestion manifest.
3. **Schedule the cloud path:** Cloud Composer hosts the Airflow DAG, which runs every two minutes with `catchup=False` and `max_active_runs=1`.
4. **Execute Task 1:** download the raw object to the task workspace, run data quality, preprocessing, and EDA, then upload processed datasets, reports, and execution logs.
5. **Execute Tasks 2 and 3:** Task 2 reads the model-ready dataset to train the Random Forest; Task 3 reads that dataset and model to produce the comparison metrics. Both upload their artifacts to Cloud Storage.

6. **Serve results:** the FastAPI layer reads pipeline artifacts and GCP runtime details, while the dashboard calls the FastAPI endpoints. The dashboard does not access Cloud Storage directly or hard-code pipeline results.

Component boundaries

Component	Responsibility	Deployment boundary
<code>src/diabetes_risk/pipeline/</code>	Ingestion, validation, preprocessing, EDA, modeling, and logging	Platform neutral; runs locally or in Airflow
<code>dags/diabetes_risk_pipeline.py</code>	Task ordering, retries, cadence, and overlap policy	Airflow / Cloud Composer
Cloud Storage	Raw, processed, report, model, and execution-log objects	GCP persistence layer
<code>src/diabetes_risk/api/</code>	FastAPI routes and built-in API clients	Application API layer
<code>src/diabetes_risk/dashboard/</code>	Activity and result presentation through API calls	Presentation layer

The API and dashboard are intentionally downstream of the pipeline outputs. They do not change the raw dataset or run preprocessing themselves. Credentials and project settings are supplied through environment configuration or Application Default Credentials; they are not embedded in the DAG or committed to the repository.

Implementation status: the pipeline, DAG, Cloud Storage hand-off, and Composer schedule are implemented and verified in Section 2. The API and dashboard are implemented, deployed, and tested; live API and dashboard evidence is recorded in Section 4. The demonstration video remains outstanding.

1. Business Understanding, Dataset, and Ingestion

1.1 Introduction

This section covers the business problem, dataset selection, and data ingestion that the rest of the pipeline (Sections 2-4) builds on. In outline, the pipeline ingests a public health survey dataset, validates and preprocesses it, runs exploratory data analysis, and re-executes automatically on a fixed schedule, with results intended to be exposed through both a cloud dashboard and a documented REST API (Swagger/OpenAPI).

1.2 Project topic

Diabetes Risk Prediction Using Health and Lifestyle Indicators

1.3 Business problem

Healthcare organizations collect large volumes of patient health and lifestyle data — clinical measurements, lifestyle habits, and demographic indicators — but converting that data into early diabetes-risk screening is difficult in practice. Manual chart-by-chart review does not scale across thousands of patient records, so at-risk individuals are often identified only after symptoms are already present or during unrelated visits, rather than through proactive screening.

This delay has real consequences: patients who could have benefited from early lifestyle intervention or monitoring progress to more advanced, harder-to-manage risk levels, and healthcare providers face higher downstream treatment costs and heavier caseloads that could have been reduced with earlier flagging.

An automated, data-driven risk-screening pipeline addresses this by continuously analyzing available health and lifestyle indicators (e.g., BMI, blood pressure, cholesterol, physical activity, general health status) and surfacing individuals whose profile suggests elevated diabetes risk — long before manual review would catch them.

Who uses the results: primarily healthcare providers and care coordinators, who can use the risk-screening output to prioritize outreach and preventive care for higher-risk patients, and secondarily program administrators, who can use the aggregated dashboard view to understand risk distribution across a patient population. The output is explicitly a **risk-screening signal**, not a diagnosis — final clinical decisions remain with a qualified provider.

1.4 Current and proposed states

Current state

Patient health and lifestyle data is already being collected by healthcare organizations and public health surveys, but it sits largely underused for proactive screening. Identifying which individuals carry elevated diabetes risk still depends on manual review of records or waiting for a clinical visit to prompt investigation, which is time-consuming and inconsistent across large patient populations. Because this analysis is not automated, insights are generated infrequently — often too late to inform early, preventive action — and there is no continuous, systematic way to flag risk as new data arrives.

Proposed state

Health and lifestyle data flows through an automated cloud-based pipeline that ingests the dataset, checks and cleans it, and analyzes diabetes-risk indicators (BMI, blood pressure, cholesterol, activity level, general health, etc.) without manual intervention. This analysis reruns on a fixed schedule, so results stay current rather than being a one-time snapshot. The outcomes — risk indicators, key drivers, and pipeline activity — are made visible through a cloud dashboard and exposed through APIs, giving healthcare providers and administrators continuous, up-to-date visibility into risk patterns instead of periodic manual reviews.

1.5 Objectives and benefits

Objectives

- Ingest and validate a real, sufficiently large public health dataset (BRFSS 2015 diabetes indicators) with automated checks on schema, target column, and row count.
- Preprocess and clean the data (missing-value handling, encoding, normalization) so it is ready for repeatable analysis.
- Run exploratory data analysis (distribution, correlation, binning, feature importance) to surface diabetes-risk indicators.
- Automate the full pipeline on a fixed (2-minute) schedule with execution logging, so results stay current without manual reruns.
- Expose pipeline results and at least four application operations through a documented, testable REST API (Swagger/OpenAPI), and visualize them on a cloud dashboard.

Benefits

- **For healthcare providers/care coordinators:** a continuously updated risk-screening signal that helps prioritize outreach to higher-risk patients, instead of relying on manual chart review.
- **For program administrators:** an aggregated dashboard view of risk distribution across a patient population, supporting resource and program planning.
- **For the team:** a working, end-to-end demonstration of the assignment's required architecture (ingestion -> preprocessing -> EDA -> scheduled DataOps -> API layer), directly mapped to the grading rubric.

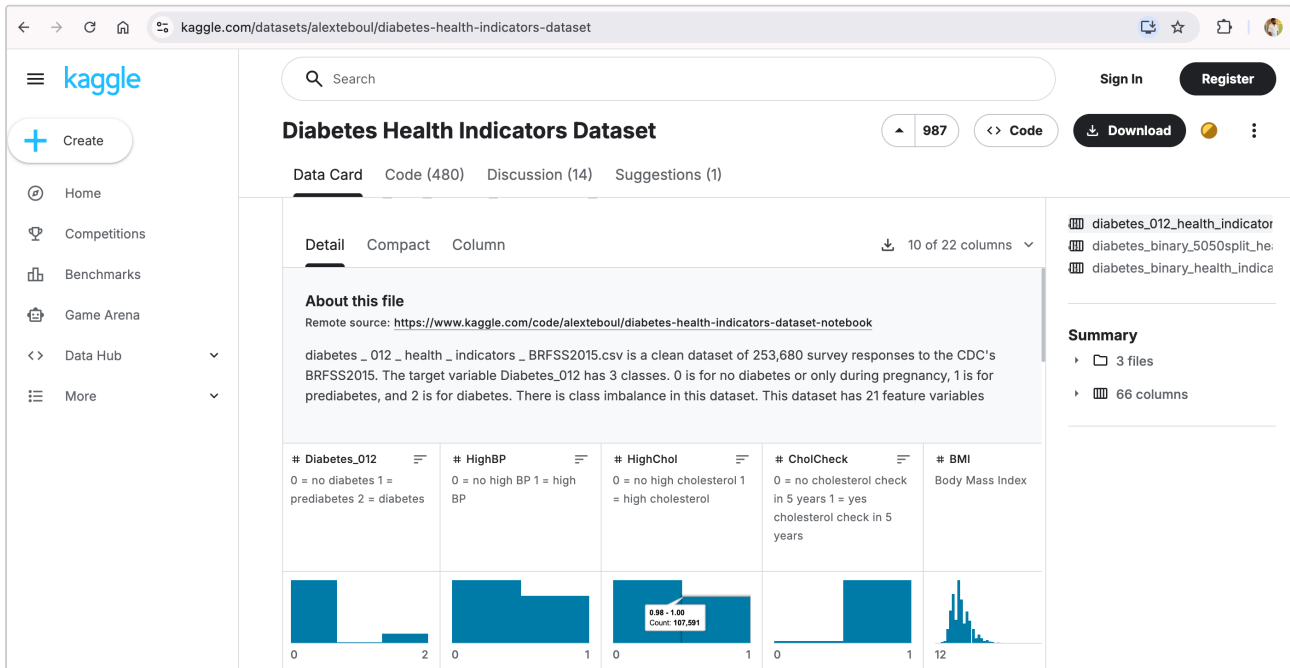
As with the rest of this report, all outputs are a **risk-screening signal**, not a clinical diagnosis.

1.6 Dataset profile

- **Filename:** `diabetes_012_health_indicators_BRFSS2015.csv`
- **Source:** Kaggle — Diabetes Health Indicators Dataset (<https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset>)
- **Row count:** 253,680
- **Column count:** 22
- **Target column:** `Diabetes_012` (0 = no diabetes, 1 = prediabetes, 2 = diabetes)
- **Feature columns:** `HighBP`, `HighChol`, `CholCheck`, `BMI`, `Smoker`, `Stroke`, `HeartDiseaseorAttack`, `PhysActivity`, `Fruits`, `Veggies`, `HvyAlcoholConsump`, `AnyHealthcare`, `NoDocbcCost`, `GenHlth`, `MentHlth`, `PhysHlth`, `DiffWalk`, `Sex`, `Age`, `Education`, `Income`
- **Data types:** All 22 columns are stored as numeric (float64); several are binary flags (0/1) or ordinal codes (e.g. `GenHlth` 1–5, `Age` bucketed 1–13, `Income` 1–8) rather than free-form categories.
- **Missing values:** 0 across all columns (independently verified against the raw CSV).
- **Target-value distribution:** 0.0 → 213,703 (84.2%), 1.0 → 4,631 (1.8%), 2.0 → 35,346 (13.9%) — notably imbalanced, especially the prediabetes class.

Why this is sufficient: At 253,680 rows, even the smallest class (prediabetes, 4,631 rows) comfortably supports EDA (correlation analysis, binning, feature importance) and, if a model is included, a held-out test split — a standard 80/20 split still leaves roughly 900+ minority-class examples for evaluation. The dataset's scale also means a full preprocessing + EDA pass is fast enough to comfortably fit inside the assignment's every-2-minute scheduled run.

Evidence -- raw dataset source (Kaggle):



Raw dataset screenshot: Kaggle "Diabetes Health Indicators Dataset" page showing the dataset title, source URL, target column (Diabetes_012) and feature definitions, and file summary

1.7 Data dictionary

All columns are stored as numeric (float64); "Role" marks the prediction target vs. feature. Ranges below are confirmed against the raw CSV, not assumed.

Column	Business meaning	Values	Role
Diabetes_012	Diabetes status	0 = no diabetes, 1 = prediabetes, 2 = diabetes	Target
HighBP	Diagnosed with high blood pressure	0 = no, 1 = yes	Feature (binary)
HighChol	Diagnosed with high cholesterol	0 = no, 1 = yes	Feature (binary)
CholCheck	Cholesterol checked in past 5 years	0 = no, 1 = yes	Feature (binary)
BMI	Body Mass Index	12–98 (continuous)	Feature (numeric)

Column	Business meaning	Values	Role
Smoker	Smoked ≥ 100 cigarettes in lifetime	0 = no, 1 = yes	Feature (binary)
Stroke	Ever told had a stroke	0 = no, 1 = yes	Feature (binary)
HeartDiseaseorAttack	Coronary heart disease or myocardial infarction	0 = no, 1 = yes	Feature (binary)
PhysActivity	Physical activity in past 30 days (excl. job)	0 = no, 1 = yes	Feature (binary)
Fruits	Consumes fruit $\geq 1 \times$ /day	0 = no, 1 = yes	Feature (binary)
Veggies	Consumes vegetables $\geq 1 \times$ /day	0 = no, 1 = yes	Feature (binary)
HvyAlcoholConsump	Heavy alcohol consumption (men > 14 , women > 7 drinks/week)	0 = no, 1 = yes	Feature (binary)
AnyHealthcare	Has any healthcare coverage	0 = no, 1 = yes	Feature (binary)
NoDocbcCost	Needed a doctor but couldn't afford it (past 12 mo.)	0 = no, 1 = yes	Feature (binary)
GenHlth	Self-rated general health	1 = excellent ... 5 = poor (ordinal)	Feature (ordinal)
MentHlth	Days of poor mental health, past 30 days	0–30 (count)	Feature (numeric)
PhysHlth	Days of poor physical health, past 30 days	0–30 (count)	Feature (numeric)
DiffWalk	Serious difficulty walking/climbing stairs	0 = no, 1 = yes	Feature (binary)
Sex	Sex	0 = female, 1 = male	Feature (binary)
Age	13-level age bracket	1 = 18–24 ... 13 = 80+ (ordinal)	Feature (ordinal)
Education	Education level	1 = none/kindergarten ... 6 = college 4+ years (ordinal)	Feature (ordinal)
Income	Household income bracket	1 = $< \$10k$... 8 = $\$75k+$ (ordinal)	Feature (ordinal)

1.8 Suitability and limitations

Why this dataset suits the assignment

- **Scale:** 253,680 rows is large enough for statistically meaningful correlation analysis, binning, and feature importance, and still leaves a workable minority-class sample (4,631 rows) if a held-out model evaluation is added.
- **Feature mix:** binary flags (`HighBP` , `Smoker` , ...), ordinal scales (`GenHlth` , `Age` , `Education` , `Income`), and continuous counts (`BMI` , `MentHlth` , `PhysHlth`) together give real material for every required EDA activity — correlation, binning, encoding, and univariate/bivariate analysis.
- **Target:** a 3-class label (`Diabetes_012`) supports classification and feature-importance work if the optional model is included, and its imbalance is itself a useful, honest talking point for the EDA/evaluation write-up.
- **Automation-friendly:** no missing values and a manageable file size (~23 MB) mean a full preprocessing + EDA pass runs quickly — important for the every-2-minute scheduled workflow.

Source and licensing

Derived from the CDC's public **Behavioral Risk Factor Surveillance System (BRFSS) 2015** survey, cleaned and republished on Kaggle by Alex Teboul (<https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset>) and also mirrored on the UCI Machine Learning Repository. The original 2015 BRFSS survey collected responses from 441,455 individuals across 330 features; this cleaned file keeps 253,680 responses and 21 feature variables plus the target.

License: CC0: Public Domain (confirmed directly on the Kaggle dataset page). No attribution or usage restriction applies, though the CDC's original BRFSS data should still be credited as the underlying source in the final report.

Limitations

- **Self-report bias:** all fields are self-reported survey answers (smoking, alcohol use, activity, etc.), which are known to be subject to recall error and social-desirability bias (e.g., under-reporting alcohol consumption or over-reporting physical activity).
- **Dataset age:** collected in 2015 — over a decade old, so absolute rates (e.g., smoking, obesity) may not reflect current populations.
- **Generalization:** BRFSS is a U.S.-only telephone survey; findings should not be generalized to non-U.S. populations, and telephone-survey methodology can under-represent some demographic groups.
- **Bias and fairness:** `Income` , `Education` , and `AnyHealthcare / NoDocbcCost` reflect healthcare-access disparities as much as biological risk — correlations involving these fields should be discussed as access/equity signals, not purely clinical ones. `Sex` is recorded as a binary field only, per the original survey design.
- **Not a diagnosis:** consistent with the rest of this report, all outputs are a **risk-screening signal**, not a clinical diagnosis, and must not be presented or used as one.

1.9 Data ingestion evidence

Implemented as `src/diabetes_risk/pipeline/ingestion.py` (`ingest()`), covering the required checks: file opens successfully, all 21 expected feature columns plus the `Diabetes_012` target are present, and the row count is recorded and compared against the verified profile (253,680). The raw CSV is never modified or duplicated -- each ingestion run instead appends a timestamped, SHA-256-hashed entry to `data/raw/ingestion_manifest.json`, so the exact file used stays verifiable and any future drift (a different file swapped in under the same name) would be visible immediately as a hash or row-count change.

Run with:

```
python -m diabetes_risk.pipeline ingest --source data/raw/
diabetes_012_health_indicators_BRFSS2015.csv --manifest data/raw/
ingestion_manifest.json
```

(The screenshot below was captured before the pipeline's commands were consolidated into one CLI, so it shows the earlier form `python -m diabetes_risk.pipeline.ingestion ...` -- the result is identical either way, since the underlying `ingest()` function didn't change.)

Verified result (first import):

```
{
  "source_path": "data/raw/diabetes_012_health_indicators_BRFSS2015.csv",
  "imported_at": "2026-09-07T17:10:57.174896+00:00",
  "sha256": "a971809d0786d2d7d7f6070f83dfe8af9d860ea33a79e0e2701c8a49f78f9861",
  "row_count": 253680,
  "column_count": 22,
  "columns_match_expected": true,
  "target_present": true,
  "row_count_matches_expected": true
}
```

Processed output (the preprocessing stage in Section 2) is written to `data/processed/`, kept separate from `data/raw/` so the original file is always preserved untouched. Covered by 5 automated tests in `tests/test_ingestion.py` (valid file, manifest append behavior, missing target column, missing feature column, missing file).

Evidence — terminal run:

```
7 passed, 2 warnings in 14.25s
(.venv) sanjaykumarpushadapu@Unknown_92:71:0b:ff:ff:73 Assignment-1 % python -m diabetes_risk.pipeline.ingestion --source data/raw/diabetes_012_health_indicators_BRFSS2015.csv --manifest data/raw/ingestion_manifest.json
{
  "source_path": "data/raw/diabetes_012_health_indicators_BRFSS2015.csv",
  "imported_at": "2026-09-08T02:25:42.732543+00:00",
  "sha256": "a971809d0786d2d7d7f6070f83dfe8af9d860ea33a79e0e2701c8a49f78f9861",
  "row_count": 253680,
  "column_count": 22,
  "columns_match_expected": true,
  "target_present": true,
  "row_count_matches_expected": true
}
(.venv) sanjaykumarpushadapu@Unknown_92:71:0b:ff:ff:73 Assignment-1 %
```

Step 7 ingestion evidence: pytest passing and ingestion command output

All 7 tests in the whole suite at that time pass (the project had 7 tests in total before later work packages added more), and the ingestion run against the real dataset matches the result recorded above exactly (same SHA-256 hash, same row/column counts), confirming it's reproducible outside the verification environment too.

In summary, the business problem, dataset profile, and data dictionary are established and verified against the real file, suitability and limitations are documented with a confirmed CC0 license, and the raw data has been ingested, validated, and logged, providing a verified foundation for the preprocessing stage in Section 2.

2. Data Quality, Preprocessing, and Pipeline Automation

2.1 Scope

This section covers **data-quality validation, preprocessing, automated EDA generation, DataOps execution, two-minute scheduling, execution logging, verification, and handoff to the EDA and modeling stage (Section 3)**. The implementation is deliberately platform-neutral so that the same Python processing code can run locally or under Apache Airflow, with Google Cloud Composer documented as the planned production orchestration environment.

The raw dataset from Section 1 remains immutable. All transformed datasets, analytical artifacts, and execution logs are written to separate `data/processed/` and `data/outputs/` locations.

2.2 Data-quality validation

A reusable validation module is implemented in `src/diabetes_risk/pipeline/data_quality.py`. The module validates the approved dataset before preprocessing and produces machine-readable quality results.

The implemented checks include:

- Schema and target-column validation
- Row and column counts
- Pandas data types
- Summary statistics
- Missing-value counts and percentages
- Exact duplicate-record detection
- Target-value validation (`Diabetes_012` must contain only 0, 1, or 2)
- Known numeric-range validation
- Binary indicator validation (0/1)

Verified source dataset: `data/raw/diabetes_012_health_indicators_BRFSS2015.csv`

Quality check	Verified result
Input records	253,680
Input columns	22
Missing values	0
Exact duplicate records	23,899 (9.42%)
Invalid target values	0
Invalid known numeric-range values	0
Invalid binary values	0
Quality status	PASS_WITH_WARNINGS

The duplicate records are treated as a **non-fatal quality warning**, rather than a schema or data-validity failure. The raw file is preserved unchanged, while exact duplicates are removed during preprocessing.

Quality artifacts generated by the pipeline are:

```
data/outputs/quality/data_quality.json
data/outputs/quality/missing_values.csv
```

2.3 Preprocessing and cleaned-data generation

The transformation contract is implemented in `src/diabetes_risk/pipeline/preprocessing.py`. The preprocessing flow is applied consistently on every pipeline execution:

1. Normalize column names to lowercase snake case for downstream processing consistency.
2. Detect and remove exact duplicate records, retaining the first occurrence.
3. Median-impute numeric missing values where required.
4. Mode-impute non-numeric values where required.
5. One-hot encode non-numeric categorical fields where required.
6. Preserve existing binary and ordinal indicators in interpretable form.
7. Keep the target separate from feature standardization.
8. Write an analytical cleaned dataset for the EDA stage (Section 3).
9. Write a separate standardized model-ready dataset for optional downstream model work.

For the current approved dataset, no imputation was required because the raw file contains **zero missing values**. Likewise, no one-hot categorical encoding was required in the current run because the dataset fields are already represented as numeric, binary, or ordinal values.

The current execution produced:

Output	Result
Input rows	253,680
Exact duplicates removed	23,899
Cleaned analytical rows	229,781
Cleaned columns	22
Model-ready rows	229,781
Model-ready columns	22

2.4 Analytical cleaned dataset vs. model-ready dataset

Two processed datasets are intentionally maintained rather than applying the same transformation to both analytical and modelling use cases.

`data/processed/diabetes_cleaned.csv` is the primary handoff to the EDA stage (Section 3). It retains the original, interpretable feature scales so that BMI, health-day counts, age groups, and other indicators remain meaningful in EDA charts and tables.

`data/processed/diabetes_model_ready.csv` is a separate downstream modelling output. Z-score standardization is applied to the continuous/count fields `BMI`, `MentHlth`, and `PhysHlth`, while binary and ordinal indicators remain interpretable and the target is not scaled.

This separation satisfies the assignment's normalization/standardization requirement without making the EDA outputs harder to interpret. It also leaves the optional model-comparison stage (Section 3.8) free to use the model-ready dataset.

The preprocessing module writes its transformation summary to:

```
data/processed/preprocessing_report.json
```

2.5 Automated EDA artifact generation

This section automates the **generation** of the EDA artifacts in `src/diabetes_risk/pipeline/eda.py`. Section 3 remains responsible for the analytical interpretation and final EDA narrative.

Each successful pipeline run refreshes the following outputs required by the workload plan:

- Summary statistics
- Missing-value summary
- Target-distribution table and chart
- Feature-distribution charts
- Correlation matrix
- Selected bivariate tables and charts

- Age/BMI binned features
- EDA metadata

The automated feature-distribution outputs cover eight selected indicators:

- BMI
- Age
- General Health
- High Blood Pressure
- High Cholesterol
- Physical Activity
- Cholesterol Check
- Smoking

The automated bivariate outputs cover six selected relationships against the diabetes target:

- BMI vs. target
- Age vs. target
- High Blood Pressure vs. target
- High Cholesterol vs. target
- Physical Activity vs. target
- General Health vs. target

This design allows the EDA artifacts to be refreshed automatically without requiring manual notebook reruns or chart regeneration downstream.

2.6 End-to-end DataOps runner

`src/diabetes_risk/pipeline/runner.py` provides the single end-to-end execution path used for both local execution and Airflow orchestration:

```
Raw CSV
  ↓
Data-quality validation
  ↓
Preprocessing
  ├── diabetes_cleaned.csv
  └── diabetes_model_ready.csv
  ↓
Automated EDA artifact generation
  ↓
Structured execution log
```

The runner records the status of each execution and writes outputs to predictable locations. The verified full-dataset local execution (run ID `20260912T025251232582Z`) started at

2026-09-12T02:52:51.232576Z , completed at 2026-09-12T02:52:56.336563Z , and finished in **5.10 seconds**. It read all **253,680** raw records and produced the expected 229,781-row processed outputs.

2.7 Verified full-dataset execution result

To remove ambiguity about the size of the data actually processed, this section retained **one fresh successful end-to-end run only** against the complete raw CSV. No two-record test fixture is represented in the retained pipeline outputs or execution results. The test suite continues to use small fixtures for unit testing, but the execution documented here is the full production-like local run against the 253,680-row source file.

Execution metric	Verified result
Run ID	20260912T025251232582Z
Input file	data/raw/diabetes_012_health_indicators_BRFSS2015.csv
Input rows read	253,680
Input columns	22
Missing values	0
Exact duplicate records detected	23,899 (9.42%)
Rows after duplicate removal	229,781
Cleaned dataset	229,781 × 22
Model-ready dataset	229,781 × 22
Invalid target values	0
Invalid known numeric-range values	0
Invalid binary values	0
Quality status	PASS_WITH_WARNINGS
Execution status	SUCCESS
Runtime	5.10 seconds

The deduplicated target distribution used by the refreshed EDA artifacts is:

Diabetes_012	Records	Percentage
0	190,055	82.71%
1	4,629	2.01%
2	35,097	15.27%
Total	229,781	100.00%

The original raw target distribution documented in Section 1 remains unchanged; the table above reflects the **post-duplicate-removal dataset** used for the refreshed EDA outputs in this section. The raw CSV itself remains immutable.

The authoritative execution record is stored at:

```
data/outputs/execution/run_20260912T025251232582Z.json
data/outputs/execution/latest_run.json
```

The generated EDA artifacts, quality report, cleaned dataset, and model-ready dataset all correspond to this same full-dataset execution.

2.8 Execution logging and monitoring information

`src/diabetes_risk/pipeline/logging_utils.py` implements structured JSON execution logging. Each pipeline run records:

- Unique run ID
- Start timestamp
- End timestamp
- Runtime duration
- Execution status
- Number of input/processed records
- Number of output records
- Duplicate count
- Errors
- Warnings
- Quality status
- Processed-output locations
- EDA-output locations

Execution artifacts are written under:

```
data/outputs/execution/run_<run_id>.json
data/outputs/execution/latest_run.json
```

For the verified final local execution (20260912T025251232582Z), the execution status was **SUCCESS** , the quality status was **PASS_WITH_WARNINGS** , **253,680** records were read and processed, **23,899** exact duplicates were detected and removed, and **229,781** records were written to both the cleaned and model-ready outputs. No missing, invalid-target, invalid-known-range, or invalid-binary values were detected.

2.9 Airflow orchestration and two-minute schedule

`dags/diabetes_risk_pipeline.py` implements the orchestration layer using **Apache Airflow**. The DAG is intentionally cloud agnostic and invokes the same platform-neutral Python runner used for local execution.

The DAG contains no GCP SDK imports, GCS-specific operators, Vertex AI operators, or cloud credentials. This preserves a clean separation between the processing application and its deployment environment.

The configured schedule is:

```
*/2 * * * *
```

Therefore, the workflow is designed to trigger **every two minutes**, including the data-quality checks, preprocessing, and automated EDA generation rather than only the ingestion step.

The reliability configuration includes:

- `catchup=False` to avoid replaying historical schedules during deployment.
- `max_active_runs=1` to prevent overlapping pipeline executions.
- One retry with a one-minute retry delay for transient task failures.

The planned production deployment target is **Google Cloud Composer**, which provides managed Apache Airflow. The Python processing modules remain platform-neutral, while `dags/diabetes_risk_pipeline.py` uses the Google Cloud Storage client to download raw inputs and upload processed datasets, reports, models, and execution logs between Composer tasks. Deployment and architecture details are documented in this report and the root README.

Cloud Composer deployment verification

The DAG was deployed to a live **Google Cloud Composer** environment to verify the two-minute schedule under real managed-orchestration infrastructure, separate from local Airflow unit testing.

Deployment detail	Value
GCP project	<code>diabetes-risk-group49</code>
Composer environment	<code>diabetes-risk-env</code>
Location	<code>us-central1</code>
Airflow version	<code>2.11.1</code>
Confirmed schedule interval (Composer console)	<code>*/2 * * * *</code>

The screenshot shows the Google Cloud Managed Airflow console for the environment 'diabetes-risk-env'. The 'DAGs' tab is selected, displaying a table of DAGs. The 'diabetes_risk_pipeline' is active and has 1 successful run in the last hour.

DAG id	State	Description	Schedule interval	Last completed run	Active runs	Successful runs (1h)	Failed runs (1h)
airflow_monitoring	Active	liveness monitoring dag	* / 10 * * * *	7 minutes ago	0	6	0
diabetes_risk_pipeline	Active	Complete diabetes ri...	* / 2 * * * *	1 minute ago	1	24	0

Cloud Composer DAG overview on 23 September 2026 showing `diabetes_risk_pipeline` Active on `* / 2 * * * *`, with 1 active, 24 successful, and 0 failed runs in the one-hour window

The Cloud Storage bucket managed by the environment is used as the data hand-off between Airflow tasks: the raw dataset is read from `data/raw/`, and processed/EDA/model outputs are written back to `data/processed/` and `data/outputs/`.

The selected Composer run, `scheduled__2026-09-23T10:14:00+00:00`, completed successfully across all three tasks: `data_quality_preprocessing_and_eda`, `random_forest`, and `model_evaluation`.

The screenshot shows the 'Run history' tab for the 'diabetes_risk_pipeline'. It displays a list of DAG runs, with the selected run 'scheduled__2026-09-23T10:14:00+00:00' expanded to show task details and logs.

Execution date (UTC)	State	Task id	Operator	Start date (UTC)	End date (UTC)	Duration	Try number	DAG
9/23/26, 10:16 AM	Success	model_evaluation	PythonOperator	9/23/26, 10:17 AM	9/23/26, 10:18 AM	1 min 19.063643 sec	1	sched
9/23/26, 10:14 AM	Success	random_forest	PythonOperator	9/23/26, 10:16 AM	9/23/26, 10:17 AM	1 min 4.026657 sec	1	sched
9/23/26, 10:10 AM	Success	data_quality_preprocessing_and_eda	PythonOperator	9/23/26, 10:16 AM	9/23/26, 10:16 AM	17.708172 sec	1	sched

Logs for DAG run: "scheduled__2026-09-23T10:14:00+00:00"

```

> [INFO] 2026-09-23 10:18:52.104 UTC airflow-worker ::group::Post task execution logs
> [INFO] 2026-09-23 10:18:52.106 UTC airflow-worker Marking task as SUCCESS. dag_id=diabetes_risk_pipeline, task_id=model_evaluation, run_id=scheduled__2026-09-23T10:14:00+00:00
> [INFO] 2026-09-23 10:18:52.301 UTC airflow-worker Task exited with return code 0
> [INFO] 2026-09-23 10:18:52.354 UTC airflow-worker 0 downstream tasks scheduled from follow-on schedule check
> [INFO] 2026-09-23 10:18:52.356 UTC airflow-worker ::endgroup::

```

Cloud Composer run history for `scheduled__2026-09-23T10:14:00+00:00` showing all three pipeline tasks in Success state

A previous one-hour console snapshot showed 10 failed runs. The later authenticated view above showed 24 successful runs, 0 failed runs, and 1 active run; these counts are time-window snapshots

and change as scheduled runs complete. The deployed `/api/v1/schedule` endpoint also returned `RUNNING` for the Composer environment. The Cloud Console page is available at <https://console.cloud.google.com/managed-airflow/environments/detail/us-central1/diabetes-risk-env/dags?project=diabetes-risk-group49>.

2.10 Configuration and local execution

Environment-driven configuration is provided through `src/diabetes_risk/pipeline/config.py`, along with a unified command-line entry point through `src/diabetes_risk/pipeline/cli.py`.

The pipeline can therefore be executed locally without GCP dependencies. The main commands are:

```
python -m diabetes_risk.pipeline ingest
python -m diabetes_risk.pipeline run
```

The default configuration uses the repository's raw dataset and separates processed and reporting outputs. Environment variables can override the dataset, target column, processed-output directory, and report-output directory when the same code is deployed to another environment.

2.11 Verification and test results

This implementation was verified using both automated tests and an end-to-end run against the real repository dataset.

Automated verification:

```
17 passed, 1 skipped
```

(This count reflects the test suite at the time this section was completed, before the model-evaluation tests in Section 3.8 were added. The full suite, including `tests/test_randomforestclassifier.py` and `tests/test_model_evaluation.py`, now passes 20 tests with 1 skipped -- see Section 3.8.)

The one skipped test is Airflow-specific because an Airflow runtime is not installed in the local development environment. Airflow is intentionally kept outside the core Python dependency set because it is expected to be supplied by the orchestration runtime, such as Cloud Composer.

Additional validation completed successfully:

- Python source compilation using `compileall`
- Repository whitespace validation using `git diff --check`
- End-to-end execution against the actual 253,680-row raw CSV
- Generated processed datasets and EDA artifacts verified at the expected output locations

2.12 Evidence summary

This report is the canonical record for the local validation and Cloud Composer evidence. The project README contains the consolidated evidence checklist.

The verified local evidence includes:

- Dataset validation
- Summary statistics and data types
- Missing-value results
- Duplicate-record results
- Invalid-value results
- Preprocessing results
- Cleaned dataset output
- Model-ready output
- Automated EDA outputs
- Structured execution log
- Airflow DAG configuration

Deployment evidence: the two Composer screenshots in Section 2.9 show the  schedule, an Active DAG with 24 successful, 0 failed, and 1 active run in the one-hour window, and a selected run with all three tasks successful. An earlier one-hour snapshot showed ten failures; the newer status is time-sensitive.

2.13 Summary

The approved dataset has been validated, quality checks and preprocessing are automated, cleaned and model-ready datasets are generated, EDA artifacts are refreshed by the workflow, structured execution logging is implemented, and the Airflow DAG is configured for the required two-minute schedule with an explicit overlapping-run policy. The latest authenticated Composer evidence shows 24 successful runs, 0 failed runs, and one active run in its one-hour window, with all three tasks successful in the selected run. The available automated verification suite passed at the time documented above. This section does **not** implement model training, model deployment, final dashboarding, or the four required application APIs; those are covered in Sections 3 and 4.

3. EDA, Optional Model, and Dashboard Analysis

3.1 Scope

This section covers interpretation of the automated EDA outputs from Section 2, plus an optional (not required by the assessment) predictive-model comparison built on top of the model-ready dataset. It

does not implement the dashboard or the four required application APIs; those are covered in Section 4.

Source: the EDA interpretation and feature/correlation charts in Sections 3.2-3.7 are drawn from [Diabetes_EDA_Person3.pptx](#) (the original analysis deck for this section). The target-distribution chart below was refreshed from the automated EDA run against the current raw CSV on 23 September 2026, after exact-duplicate removal; its values match the 229,781-row analytical dataset documented in this report. Section 3.8's model implementation, evaluation, and evidence were independently run and verified for this report (see Section 3.9 for the reproducibility distinction between the two).

3.2 Dataset overview and completeness

The interpretation is based on the analytical dataset produced in Section 2 after exact-duplicate removal:

Metric	Value
Analyzed records	229,781
Variables	22
Target classes	3 (Diabetes_012)
Missing values	0 (0.0% across every field)

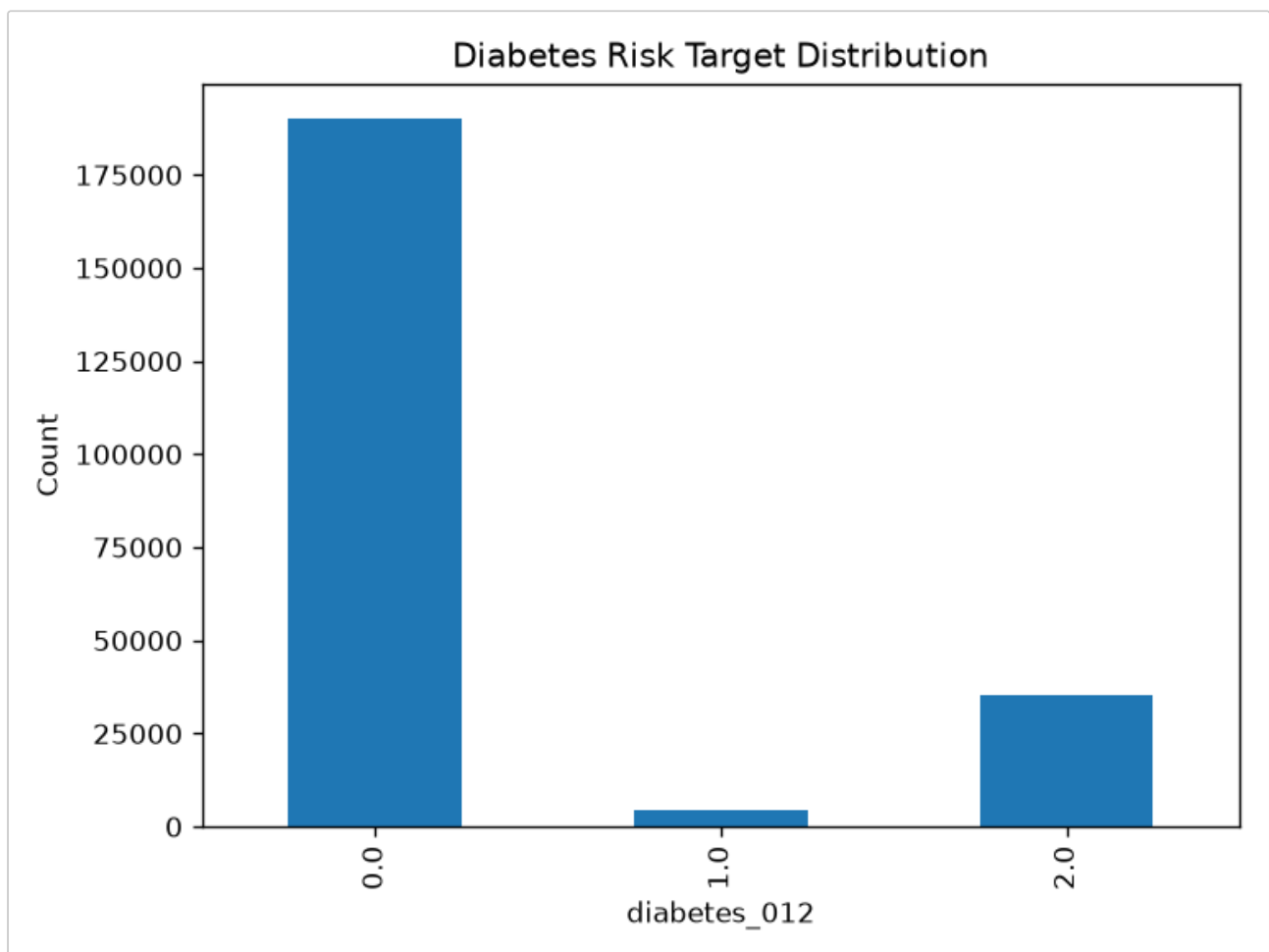
The 22 features group into four categories: clinical (high BP, high cholesterol, stroke, heart disease), lifestyle (physical activity, smoking, fruit/vegetable intake), health status (BMI, general/mental/physical health, mobility), and demographic (age band, education, income, sex).

Because the missing-value summary reports zero missing observations across all 22 analyzed variables, no rows required exclusion for missingness at this stage; this conclusion applies specifically to the processed analytical dataset used for these outputs.

3.3 Target distribution

The target is strongly imbalanced, particularly for the prediabetes class:

Diabetes_012	Records	Percentage
0 — No diabetes	190,055	82.71%
1 — Prediabetes	4,629	2.01%
2 — Diabetes	35,097	15.27%

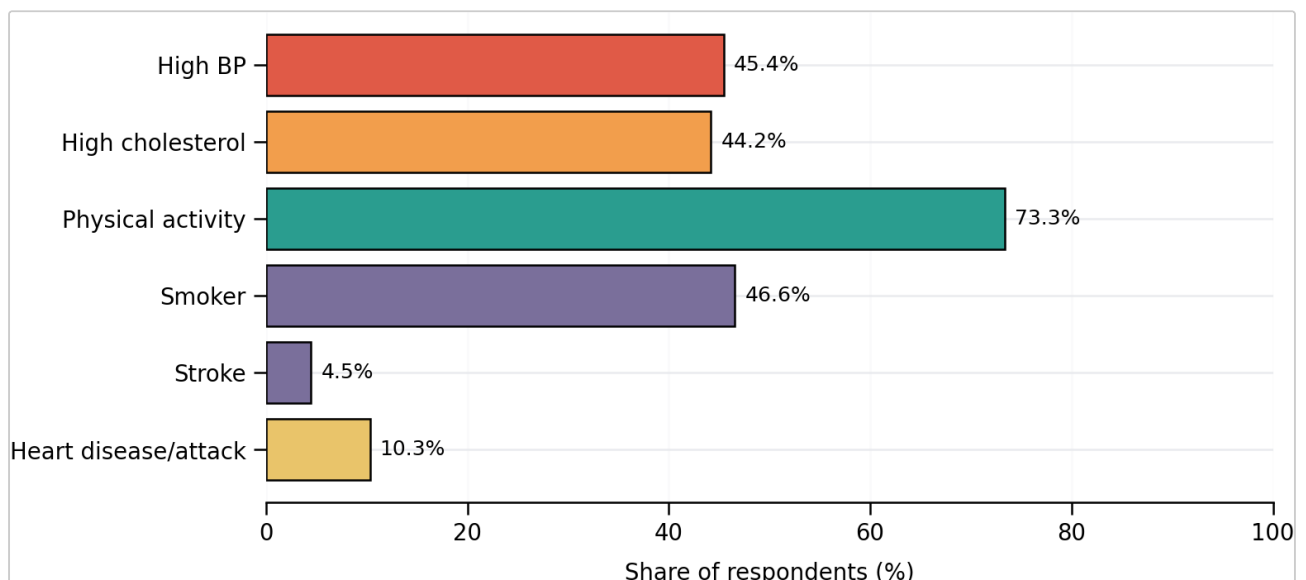


Refreshed diabetes risk target distribution from the 229,781-row cleaned dataset: 190,055 records for class 0, 4,629 for class 1, and 35,097 for class 2

Modeling implication: with this degree of imbalance, accuracy alone is not a reliable measure of model quality. Class-level precision, recall, and F1-score must be reported alongside it — this is confirmed empirically in Section 3.8 below.

3.4 Feature distribution analysis

The population combines high cardiometabolic exposure with widespread reported physical activity:



Feature distribution: share of respondents reporting High BP (45.4%), High cholesterol (44.2%), Physical activity (73.3%), Smoker (46.6%), Stroke (4.5%), and Heart disease/attack (10.3%)

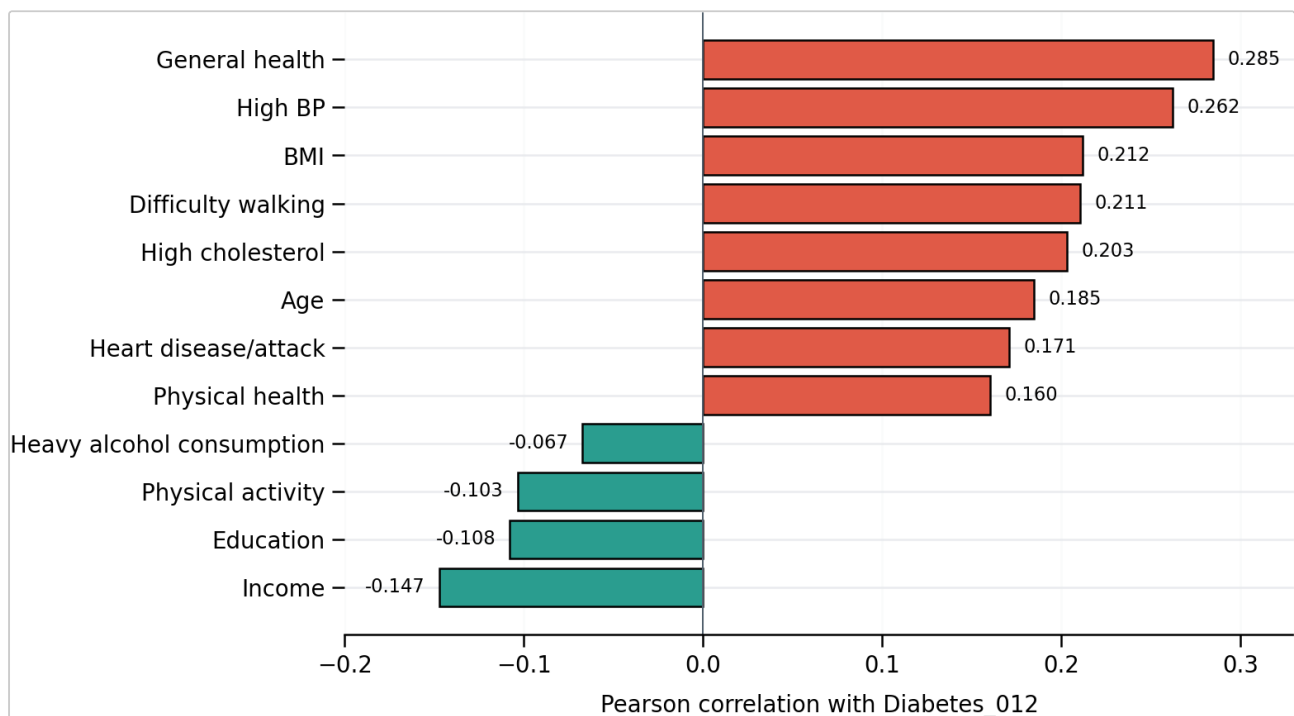
Selected distribution statistics:

Feature	Mean	Median	Range / Scale
BMI	28.69	27	12 to 98
Age band	8.09	8	1 to 13
General health	2.60	3	1 to 5

High BP and high cholesterol each affect roughly 4 in 10 respondents (45.4% and 44.2% respectively), while 73.3% report physical activity, 46.6% report smoking, 10.3% report heart disease/attack, and 4.5% report a prior stroke.

3.5 Correlation analysis

Associations with `Diabetes_012` are modest; no single feature dominates:



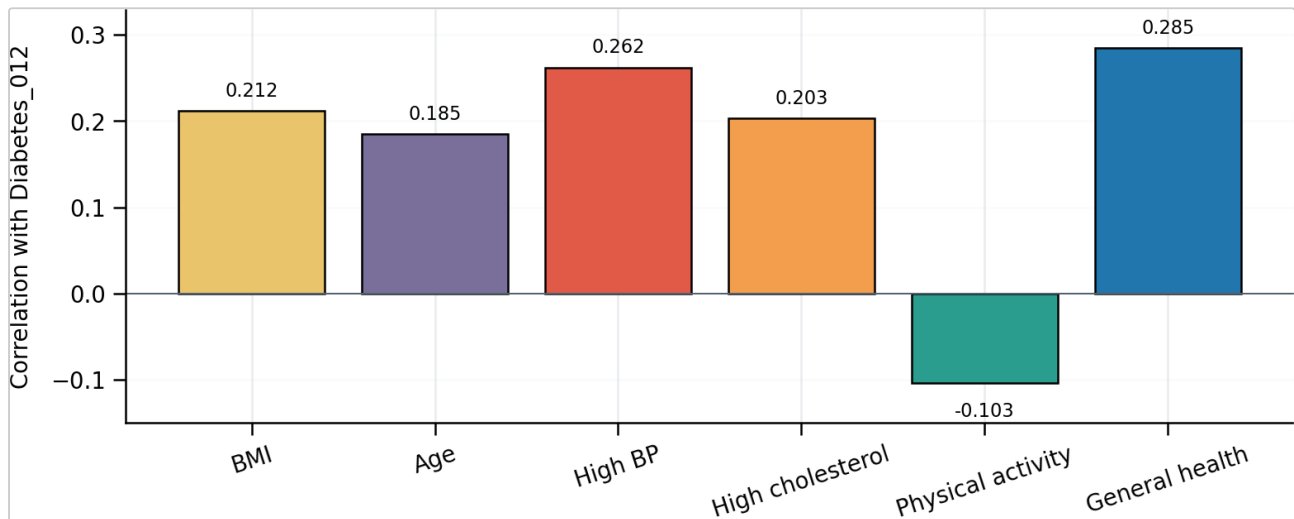
Pearson correlation of 12 features with Diabetes_012, ranging from +0.285 (General health) to -0.147 (Income)

Strongest positive	r	Strongest negative	r
General health	+0.285	Income	-0.147
High BP	+0.262	Education	-0.108
BMI	+0.212	Physical activity	-0.103
Difficulty walking	+0.211	Heavy alcohol consumption	-0.067
High cholesterol	+0.203		

Correlation indicates association, not causation.

3.6 Bivariate analysis

Six priority features were compared against the ordered diabetes outcome:



Bivariate analysis: correlation of BMI, Age, High BP, High cholesterol, Physical activity, and General health with Diabetes_012

General health has the strongest of the six associations (+0.285). High BP (+0.262), BMI (+0.212), and high cholesterol (+0.203) all rise with the ordered target, and age is positively associated (+0.185). Physical activity is inversely associated (-0.103).

Interpretation boundary: these values summarize linear association; they do not prove a causal effect or substitute for group-level significance tests.

3.7 Consolidated EDA findings

- **Data quality:** 229,781 complete records; zero missing values across all 22 analyzed fields.
- **Target risk:** 82.71% of records are class 0; prediabetes is only 2.01%, creating severe class imbalance.
- **Health signals:** general health, high BP, BMI, mobility difficulty, and high cholesterol show the largest positive associations with the target.
- **Protective associations:** income, education, and physical activity show the largest negative correlations with the target.

3.8 Optional model implementation and evaluation

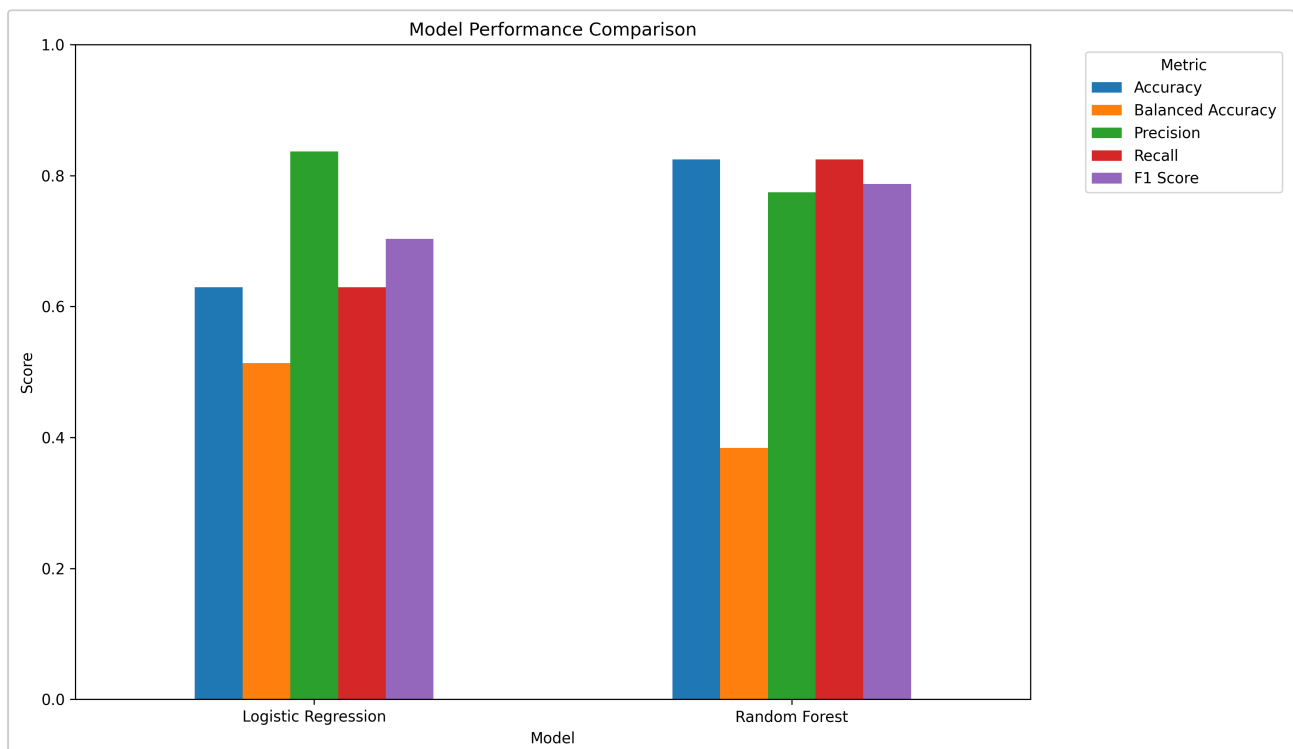
An optional predictive-model comparison was implemented on top of the model-ready dataset, even though the assessment does not require a prediction model (see the Workload Plan's Project Details). This is additive, value-added analysis rather than a required deliverable.

Implementation: `src/diabetes_risk/pipeline/randomforestclassifier.py` trains a Random Forest (200 trees, `random_state=42`) on an 80/20 stratified train/test split (183,824 train / 45,957 test records) and saves feature importances. `src/diabetes_risk/pipeline/model_evaluation.py` reproduces the same split, trains a Logistic Regression model (`class_weight="balanced"`, `StandardScaler` -normalized features), and evaluates both models side by side. Both are exposed as CLI subcommands (`python -m diabetes_risk.pipeline.randomforestclassifier` and `python -m`

`diabetes_risk.pipeline model_evaluation`) and as separate Cloud Composer DAG tasks (Section 2.9).

Verified local run result (executed against the real 229,781-row model-ready dataset):

Model	Accuracy	Balanced Accuracy	Precision (weighted)	Recall (weighted)	F1 (weighted)
Logistic Regression	62.92%	51.35%	83.69%	62.92%	70.33%
Random Forest	82.47%	38.39%	77.42%	82.47%	78.74%



Model performance comparison: grouped bar chart of Accuracy, Balanced Accuracy, Precision, Recall, and F1 Score for Logistic Regression and Random Forest

Random Forest has the higher raw accuracy but the lower balanced accuracy — this is the class-imbalance effect flagged in Section 3.3, and it shows up directly in the per-class results. Random Forest's classification report scores **0.0 precision, recall, and F1 for class 1 (prediabetes)** — it never predicts the minority class at all (confusion matrix: 0 of 926 true prediabetes cases predicted correctly). Logistic Regression, trained with `class_weight="balanced"`, trades off overall accuracy for materially better minority-class handling: 30.3% recall on class 1 and 59.3% recall on class 2, at the cost of much lower precision on class 0.

Random Forest confusion matrix	Pred 0	Pred 1	Pred 2
True 0	36,568	55	1,389
True 1	820	0	106

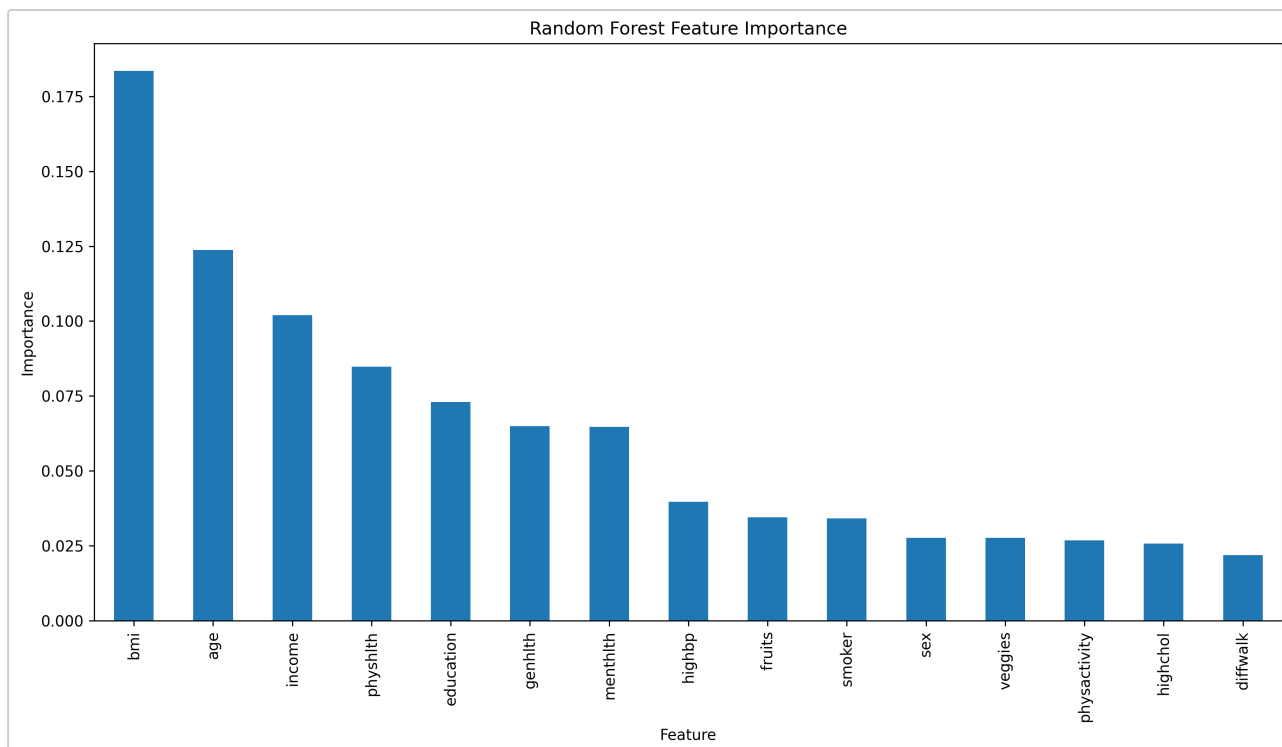
Random Forest confusion matrix	Pred 0	Pred 1	Pred 2
True 2	5,682	6	1,331

Logistic Regression confusion matrix	Pred 0	Pred 1	Pred 2
True 0	24,469	6,729	6,814
True 1	265	281	380
True 2	1,206	1,649	4,164

Conclusion: neither model is production-ready for the minority prediabetes class, and accuracy alone would have hidden this — Random Forest's 82.5% accuracy looks strong until the per-class breakdown shows it never identifies a single prediabetes case. This confirms the EDA's own modeling-implication warning (Section 3.3) with actual model results rather than assumption.

Covered by automated tests in `tests/test_randomforestclassifier.py` (verifies the model artifact, feature-importance CSV/plot, and that the target column is excluded from feature importances) and `tests/test_model_evaluation.py` (verifies both models train, all comparison/report/confusion-matrix artifacts are written, and that evaluation fails clearly if the Random Forest model has not been trained first). Both use synthetic fixtures via pytest's `tmp_path`, matching the existing test style; the full suite passes 20 tests with 1 skipped (Section 2.11).

Feature importance (Random Forest, top 10 of 21):



Random Forest feature importance: BMI highest at 0.183, followed by Age, Income, Physical health, Education, General health, Mental health, High BP, Fruits, and Smoker

Feature	Importance
BMI	0.183
Age	0.124
Income	0.102
Physical health (days)	0.085
Education	0.073
General health	0.065
Mental health (days)	0.065
High BP	0.040
Fruits	0.035
Smoker	0.034

Note that Random Forest's feature-importance ranking (BMI, age, income) differs from the EDA's linear-correlation ranking (general health, high BP, BMI) — this is expected, since feature importance captures non-linear and interaction effects the Pearson correlation in Section 3.5 does not.

3.9 Known limitations

- The feature-distribution, correlation, and bivariate charts in Sections 3.4-3.6 remain static images from the standalone Person 3 analysis deck; no checked-in script regenerates those exact presentation charts. The target-distribution chart in Section 3.3 was refreshed from the automated EDA pipeline on 23 September 2026. The model-evaluation charts and metrics in Section 3.8 were independently regenerated and are reproducible via the two CLI commands listed above.
- The optional model (Section 3.8) is not integrated into the automated `run()` pipeline or DAG Task 1 from Section 2 — it runs as separate CLI commands / DAG tasks, consistent with it being optional, additive analysis rather than part of the required DataOps pipeline.

4. Dashboard, APIs, and Demonstration

4.1 Activity dashboard

The dashboard is served by `src/diabetes_risk/dashboard/app.py` and reads all application values from the FastAPI service; it does not embed run or dataset results. It displays the most recent pipeline status and runtime, processed records, duplicate removals, quality status, error and warning counts, recent GCS-backed execution history, and optional model-comparison metrics. Its API-derived details identify the endpoint used and show the latest refresh time.

Run the API and dashboard in separate terminals after configuring GCP access:

```
uvicorn diabetes_risk.api.main:app --reload --host 127.0.0.1 --port 9000
uvicorn diabetes_risk.dashboard.app:app --reload --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000/` for the dashboard and `http://127.0.0.1:9000/docs` for Swagger/OpenAPI. The dashboard's four required application details map to the following endpoints:

API	Purpose	Method	Data source	Authentication / expected status
Workflow	Recent pipeline executions	GET <code>/api/v1/workflow</code>	Cloud Composer Airflow REST API (IAP-authenticated)	Runtime service-account impersonation for an IAP identity token; 200 when the Airflow web server is reachable
Latest execution	Run status, duration, counts, errors, warnings	GET <code>/api/v1/runs/latest</code>	GCS <code>data/outputs/execution/latest_run.json</code>	Application ADC; 200 when object exists
Dataset processing	Row/column counts, duplicates, missing values, quality status	GET <code>/api/v1/dataset</code>	GCS quality and preprocessing JSON reports	Application ADC; 200 when objects exist
Schedule/deployment	Composer environment state and Airflow version	GET <code>/api/v1/schedule</code>	Cloud Composer Environments API	Application ADC; 200 when authorized
Optional model detail	Comparison metrics and top feature	GET <code>/api/v1/model</code>	GCS model-evaluation and feature-importance CSVs	Application ADC; 200 when objects exist

`/health` and `/api/v1/metadata` are also available without GCP access. The Airflow service client and GCS artifact parsers are covered by mocked unit tests; `tests/test_api.py` verifies that the workflow, latest-run, dataset, schedule, and model routes return successful responses with those service results. Workflow history is read from the Airflow REST API via IAP-authenticated service-account impersonation, so it depends on Composer web-server identity access being correctly configured; the other three required endpoints (latest run, dataset, model) read Cloud Storage directly and do not share this dependency.

4.2 Live verification and evidence

The deployed dashboard and API were opened and tested through the public Cloud Run URLs on 23 September 2026. The dashboard loaded live API values and displayed the latest successful run, dataset metrics, Composer state, and ten recent executions. Swagger `Try it out` requests were executed against the deployed API; all four required application-detail endpoints returned HTTP 200. These are live cloud responses, not mocked test results.

Deployed services:

- Dashboard: <https://diabetes-risk-dashboard-573458509120.us-central1.run.app/>
- Swagger/OpenAPI: <https://diabetes-risk-api-573458509120.us-central1.run.app/docs>

API	Purpose	Method and endpoint	Authentication	Live response/status	Evidence
Workflow	Recent execution history	GET https://diabetes-risk-api-573458509120.us-central1.run.app/api/v1/workflow	Public Cloud Run URL; no credentials prompted	200; 10 runs; newest was 20260923T094129650583Z , success	 Swagger workflow request and response; full request URL is visible in Swagger
Latest execution	Detailed latest run status and metrics	GET https://diabetes-risk-api-573458509120.us-central1.run.app/api/v1/runs/latest	Public Cloud Run URL; no credentials prompted	200; 20260923T094129650583Z , SUCCESS , 253,680 input records	 Swagger latest-run request and response; full request URL is visible in Swagger
Dataset processing	Dataset and quality metrics	GET https://diabetes-risk-api-573458509120.us-central1.run.app/api/v1/dataset	Public Cloud Run URL; no credentials prompted	200; 253,680 input rows, 229,781 output rows, 23,899 duplicates removed, 0 missing	

API	Purpose	Method and endpoint	Authentication	Live response/status	Evidence
		<code>central1.run.app/api/v1/dataset</code>	credentials prompted	values after processing, <code>PASS_WITH_WARNINGS</code>	 Swagger dataset request and response; full request URL is visible in Swagger
Schedule/ deployment	Composer environment status and version	GET <code>https://diabetes-risk-api-573458509120.us-central1.run.app/api/v1/schedule</code>	Public Cloud Run URL; no credentials prompted	200; <code>diabetes-risk-env</code> is <code>RUNNING</code> ; Composer 3 / Airflow 2.11.1	 Swagger schedule request and response; full request URL is visible in Swagger

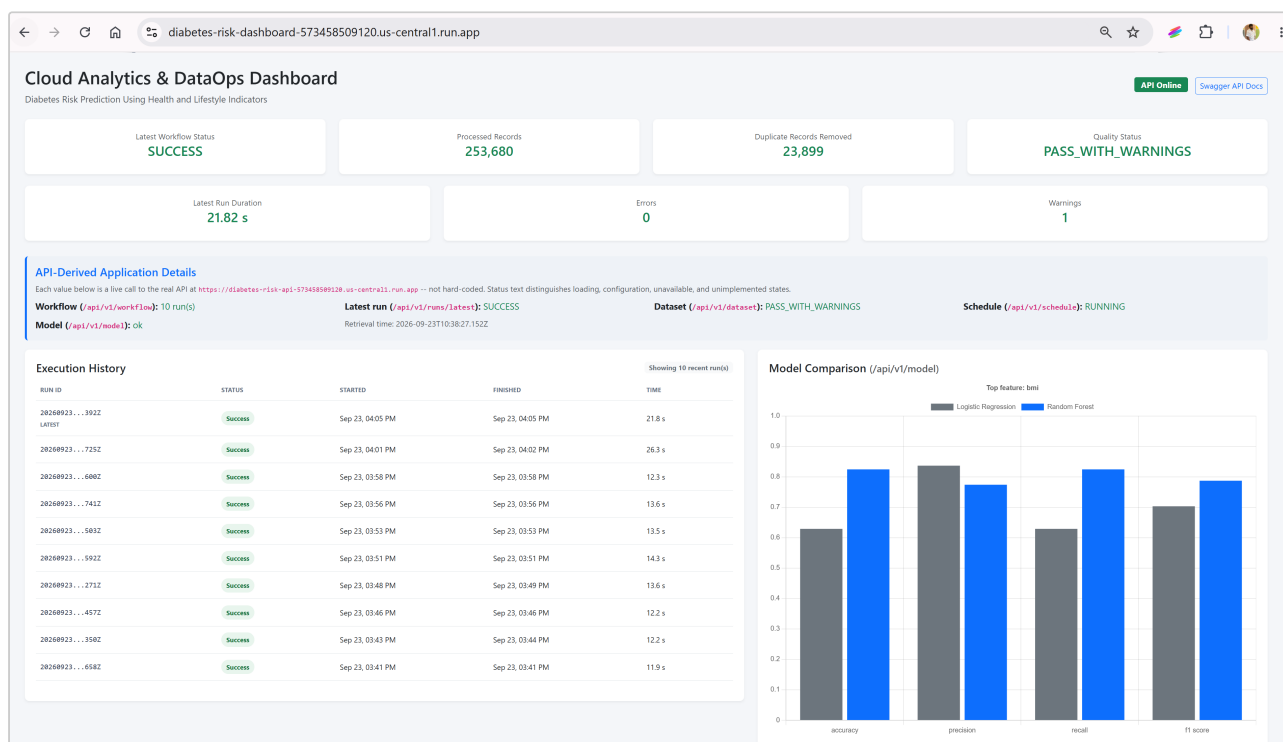
4.3 Team contribution summary

This table records the original work-package contributions and completion status documented for the project, distinct from the revised, equal-effort planning proposal in the workload plan.

Team member	Student email / ID	Contribution and status
Person 1: Pushadapu Sanjay Kumar	<code>2025ae05898@wilp.bits-pilani.ac.in</code>	Developed the business understanding, verified and profiled the dataset, prepared the data dictionary, and completed raw-data ingestion and validation.

Team member	Student email / ID	Contribution and status
Person 2: Sathish Krishnan V	2025ae05425@wilp.bits-pilani.ac.in	Implemented data-quality checks, preprocessing, pipeline automation, execution logging, and the two-minute Cloud Composer workflow; deployment evidence is documented in Section 2.9.
Person 3: Chezrla Raga Suma	2025ae05828@wilp.bits-pilani.ac.in	Completed EDA interpretation and visual analysis, feature-importance analysis, the optional Random Forest vs. Logistic Regression comparison, and dashboard analytics handoff.
Person 4: Bhuvnesh Mishra	2025ae05391@wilp.bits-pilani.ac.in	Implemented and deployed the dashboard and API, tested and documented the four required API details and additional model endpoint, and captured live evidence. The demonstration video remains outstanding.

The dashboard screenshot records the deployed view, including the API-derived application details and ten-row Execution History table. Its source page is <https://diabetes-risk-dashboard-573458509120.us-central1.run.app/>:



Deployed Cloud Run dashboard with live metrics and recent execution history

The API and dashboard screenshot evidence above is current as of 23 September 2026. The end-to-end demonstration video is the one item from Section 4 not yet captured; video upload, contribution sign-off, and submission logistics are tracked in the team's workload plan rather than in this report.